Vidyalekhan
DATE
PAGE

JAVA INTERVIEW QUESTIONS

1. What is Java ? Explain.

Java is a high-level, object-oriented, class-based programming language developed by Sun Microsystems. It is designed to be simple, secure, robust and platform independent.

Key points :

- Object-Oriented - supports classes, objects, inheritance, polymorphism, abstraction and encapsulation.
- Platform Independent - Java code is compiled into bytecode, which runs on the JVM.
- Secure - provides features such as bytecode verification and no direct pointer access.
- Robust - strong memory management and exception handling.

Example :

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

2. Why is Java platform independent ?

Because Java source code is compiled into bytecode that can run on any system having a compatible JVM.

Ajeet Yadav

3. **Differentiate between JDK, JRE and JVM ?**

JVM : Java Virtual Machine. It executes bytecode.
JRE : Java Runtime Environment. It includes JVM + libraries to run Java applications.
JDK : Java Development Kit. It includes JRE + development tools like javac, java, debugger etc.

4. **What is the main() method in Java ?**

The main() method is the entry point of a Java application. JVM starts the execution of the program from main().

Syntax :

```
public static void main(String[] args) {
    // code to be executed
}
```

5. **What are the features of Java ?**

- Simple and easy to learn
- Object - Oriented
- Platform Independent (Write Once, Run Anywhere)
- Secure
- Robust and Reliable
- Multithreaded
- Automatic Memory Management (Garbage Collection)
- High Performance
- Distributed
- Dynamic and Extensible

6. **What is a class in Java ?**

A class is a blueprint or template that defines the properties (variables) and behaviors (methods) of an object.

Example :

```
class Student {
    int id ;
    String name ;
}
```

7. **What is an object in Java ?**

An object is a real-world entity with state (data) and behavior (methods). It is an instance of a class.

Example :

```
class Student {
    String name;
    void display() {
        System.out.println(name);
    }
}
Student s = new Student();  // object creation
```

8. **What is constructor in Java ?**

A constructor is a special method that is automatically called when an object is created. It is used to initialize the object.

- Same name as class
- No return type
- Can be overloaded

Example :

```
class Student {
    Student() {
        System.out.println("Constructor called");
    }
}
```

9. **What is method overloading ?**

Method overloading means having multiple methods with the same name but different parameters in the same class.

Example :

```
class Calculator {
    int add(int a, int b) { return a+b; }
    double add(double a, double b) { return a+b; }
}
```

10. **What is method overriding ?**

Method overriding means a subclass provides its own implementation of a method that is already defined in its superclass.

11. **What is inheritance in Java ?**

Inheritance allows a class to acquire the properties and methods of another class. The existing class is called parent (super class) and the new class is called child (sub class). It helps in code reusability.
Inheritance is achieved using the extends keyword.

Example :

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

12. **What is polymorphism in Java ?**

Polymorphism means one interface having multiple forms. The same method can behave differently in different situations. It is of two types :

1. Compile time polymorphism (Method Overloading)
2. Runtime polymorphism (Method Overriding)

Example (Method Overloading) :

```
class Test {
    void show() { System.out.println("No argument"); }
    void show(int a) { System.out.println("Value:"+a); }
}
```

13. **What is encapsulation in Java ?**

Encapsulation is wrapping data (variables) and methods into a single unit (class) and restricting direct access to the data using access modifiers like private.
It helps in data hiding and provides better control.

Example :

```
class Student {
    private int id;
    public void setId(int id) { this.id = id; }
    public int getId() { return id; }
}
```

14. **What is abstraction in Java ?**

Abstraction means hiding the implementation details and showing only the essential features to the user. It can be achieved using abstract class and interface.
It helps in reducing complexity and increasing security.

15. What is an interface in Java ?

An interface in Java is a reference type that defines a contract of methods (without implementation). A class implements an interface using the implements keyword and provides the method definitions.

Example :

```
interface Animal {
    void sound();  // abstract method
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}
```

16. Difference between abstract class and interface ?

Abstract Class	Interface
• Can have abstract and non-abstract methods.	• Only abstract methods (from Java 8, default and static methods also allowed).
• Can have instance variables.	• Only constants (public static final).
• Can have constructors.	• Cannot have constructors.
• Supports single inheritance (one class can extend one abstract class).	• Supports multiple inheritance (a class can implement multiple interfaces).
• Used when classes share common behavior.	• Used to achieve full abstraction and define a contract.

17. What is the final keyword in Java ?

The final keyword is used to restrict modification.

- final variable – value cannot be changed.
- final method – cannot be overridden.
- final class – cannot be inherited.

Example :

```
final int x = 10;                // final variable
class A {
    final void show() { System.out.println("Final method"); }
}
final class B {                  // cannot be extended
```

18. What is the this keyword in Java ?

The this keyword refers to the current object of a class. It is used to access current class members and resolve name conflicts between instance variables and parameters.

Example :

```
class Student {
    int id;
    String name;
    Student(int id, String name) {
        this.id = id;        // refers to current object
        this.name = name;
    }
}
```

19. **What is static keyword in Java ?**

The static keyword is used to make a member (variable, method or block) belong to the class rather than to an object. Static members are shared among all objects of the class and can be accessed without creating an object.

Example :

```
class Student {
    static String college = "ABC College";
    static void show() {
        System.out.println("Static method");
    }
}
public class Test {
    public static void main(String[] args) {
        System.out.println(Student.college);
        Student.show();
    }
}
```

20. **What is this() and super() in Java ?**

- this() is used to call the current class constructor. It is used for constructor chaining (within the same class).
- super() is used to call the parent class constructor. It is used for constructor chaining (from superclass).

Example :

```
class A {
    A() { System.out.println("Parent constructor");}
}
class B extends A {
    B() {
        super();  // calls parent constructor
        System.out.println("Child constructor");
    }
}
```

21. **What is the difference between this and super ?**

this	super
• Refers to current class object.	• Refers to parent class object.
• Used to access current class members.	• Used to access parent class members.
• Can be used to call current class constructor this().	• Can be used to call parent class constructor super().
• Cannot be used to access parent class members.	• Cannot be used to access current class members.

22. **What is a package in Java ?**

A package is a collection of related classes and interfaces. It helps in organizing code, avoiding name conflicts and providing access control.

Example :

```
package com.example;      // package declaration
class Demo { }
```

23. **What is an exception in Java ?**

An exception is an abnormal event that occurs during the execution of a program. It disrupts the normal flow of the program. Java provides a mechanism to handle exceptions using try, catch, finally and throw / throws keywords.

Example :

```
try {
    int a = 10 / 0;      // ArithmeticException
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
} finally {
    System.out.println("Finally block executed");
}
```

24. **What is the difference between checked and unchecked exceptions ?**

Checked Exception	Unchecked Exception
• Checked at compile time.	• Checked at runtime.
• Must be handled using try-catch or declared using throws.	• Not required to handle or declare.
• It is a subclass of Exception (except RuntimeException).	• It is a subclass of RuntimeException.
• Examples : IOException, SQLException, ClassNotFoundException.	• Examples : ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException.

25. **What is garbage collection in Java ?**

Garbage collection is an automatic memory management process in Java. It removes objects that are no longer in use and frees memory. It is handled by the JVM.

Example :

```
class Test {
    public static void main(String[] args) {
        Test t1 = new Test();
        t1 = null;    // Eligible for garbage collection
        System.gc();  // Request JVM to run garbage collector
    }
}
```

26. **What is the difference between == and equals() in Java ?**

== operator	equals() method
• Used to compare references (memory addresses).	• Used to compare content of objects.
• Returns true only if both references point to the same object.	• Defined in Object class and can be overridden.
	• Returns true if the content of both objects is same.

27. What is String, StringBuffer and StringBuilder in Java ?

String is an immutable class, meaning once a String object is created its value cannot be changed.
StringBuffer and StringBuilder are mutable classes, used when we need to modify the string multiple times.

String	StringBuffer	StringBuilder
• Immutable	• Mutable	• Mutable
• Slow (because new object is created)	• Synchronized (thread-safe)	• Not synchronized (not thread-safe)
• Thread-safe not required (immutable)	• Slower than StringBuilder	• Faster than StringBuffer
• Used when data is constant	• Used in multi-threaded environment	• Used in single-threaded environment

28. What is the difference between Array and ArrayList ?

Array	ArrayList
• Fixed size.	• Dynamic size (can grow and shrink).
• Can store both primitive and object types.	• Can store only object types (wrapper classes for primitives).
• Part of core Java (not a collection framework).	• Part of Java Collection Framework (java.util package).
• Length is defined at the time of creation.	• Size can be increased or decreased dynamically.

29. What is the difference between HashSet, LinkedHashSet and TreeSet ?

HashSet	LinkedHashSet	TreeSet
• Does not maintain insertion order.	• Maintains insertion order.	• Maintains elements in sorted order.
• Uses hash table implementation.	• Uses hash table + linked list.	• Uses Red-Black tree implementation.
• Allows only unique elements.	• Allows only unique elements.	• Allows only unique elements.
• Generally faster.	• Slightly slower than HashSet.	• Slower than HashSet and LinkedHashSet.

30. What is the difference between HashMap and Hashtable ?

HashMap	Hashtable
• Not synchronized (not thread-safe).	• Synchronized (thread-safe).
• Allows one null key and multiple null values.	• Does not allow null key or null values.
• Introduced in Java 1.2.	• Introduced in Java 1.0.
• Faster than Hashtable.	• Slower than HashMap.

31. What is the difference between Vector and ArrayList ?

Vector	ArrayList
• Synchronized (thread-safe).	• Not synchronized (not thread-safe).
• Slower than ArrayList.	• Faster than Vector.
• Introduced in Java 1.0.	• Introduced in Java 1.2.
• All methods are synchronized.	• Only needed methods can be synchronized manually.
• Used in multi-threaded environment.	• Used in single-threaded environment.

32. What is fail-fast and fail-safe in Java Collections ?

Fail-fast

- Throws ConcurrentModificationException if collection is modified while iterating.
- Used in most of the collection classes (like ArrayList, HashMap).
- It prevents inconsistent state.

Fail-safe

- Does not throw ConcurrentModificationException.
- Works on a copy of the original collection.
- Used in classes like CopyOnWriteArrayList, ConcurrentHashMap.
- Slower than fail-fast.

33. What is autoboxing and unboxing in Java ?

Autoboxing is the automatic conversion of a primitive type to its corresponding wrapper class object.

Unboxing is the automatic conversion of a wrapper class object to its corresponding primitive type.

Example :

```
Integer a = 10;   // autoboxing (int -> Integer)
int b = a;        // unboxing (Integer -> int)
```

34. What is enum in Java ?

Enum is a special data type used to define a fixed set of constants. It provides type safety and better code readability.

Example :

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY;
}
class Test {
    public static void main(String[] args) {
        Day d = Day.MONDAY;
        System.out.println(d);
    }
}
```

35. **What is multithreading in Java ?**

Multithreading is a feature in Java that allows multiple threads to run concurrently within a single process. It helps in better CPU utilization and improves the performance of an application.

Example :

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();   // starts a new thread
    }
}
```

36. **What is the difference between Runnable and Thread ?**

Runnable	Thread
• It is an interface.	• It is a class.
• Contains only run() method.	• Provides run() method and other methods like start(), sleep().
• Does not provide its own thread, we need to pass it to a Thread class.	• Creates its own thread.
• Supports multiple inheritance (since it is an interface).	• Does not support multiple inheritance.
• Recommended for better design.	• Less flexible than Runnable.

37. **What is synchronization in Java ?**

Synchronization is a mechanism used to control access to a shared resource by multiple threads. It ensures that only one thread can execute a block of code at a time, preventing data inconsistency.

Example :

```
class Counter {
    int count = 0;
    synchronized void increment() {
        count++;
    }
}
```

38. **What is a constructor in Java ?**

A constructor is a special method that is automatically called when an object of a class is created. It is used to initialize the object.

Key points :

- Constructor name is same as class name.
- It does not have a return type (not even void).
- It can be overloaded.
- If no constructor is defined, a default constructor is provided by JVM.

39. **What is method overloading in Java ?**

Method overloading means having multiple methods with the same name in the same class but with different parameters (different number, type or order of parameters). It helps in code reusability.

Example :

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b +c;
    }
}
```

40. **What is method overriding in Java ?**

Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its parent class. It is used for runtime polymorphism.

Example :

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

41. **What is abstraction in Java ?**

Abstraction is the process of hiding the implementation details and showing only the essential features to the user. It can be achieved using abstract classes and interfaces.

42. **What is an abstract class and abstract method ?**

- An abstract class cannot be instantiated.
- It can have both abstract and non-abstract methods.
- An abstract method is a method without a body (no implementation).
- It must be implemented by the subclass.

43. **What is an interface in Java ?**

- An interface is a blueprint of a class.
- It contains only abstract methods (till Java 7).
- From Java 8, it can also have default and static methods.
- A class implements an interface using the implements keyword.

44. What is the final keyword in Java ?

The final keyword is used to restrict modifications. It can be used with variables, methods and classes.

- final variable → cannot be changed (constant).
- final method → cannot be overridden.
- final class → cannot be inherited.

Example :

```
final int SPEED = 100;  // final variable
```

```
class A {
    final void show() {
        System.out.println("Final method");
    }
}
```

45. What is the static keyword in Java ?

The static keyword is used for memory management. It belongs to the class rather than an object. It can be used with variables, methods and blocks.

- static variable → shared by all objects.
- static method → can be called without creating an object.
- static block → used to initialize static data.

Example :

```
class Test {
    static int count = 0;   // static variable

    static void show() {   // static method
        System.out.println("Hello");
    }
}
```

```
class Demo {
    static {        // static block
        System.out.println("Static block
                            executed");
    }
}
```

46. What is the difference between final, finally and finalize ?

Keyword	Type	Purpose	Usage
final	Keyword	Restricts modification (variable, method, class)	final int x = 10;
finally	Block	Executes always (used with try-catch)	try { } catch() { } finally { }
finalize()	Method	Called by Garbage Collector before object is destroyed (deprecated in modern Java)	protected void finalize() { }

47. What is the difference between this and super keyword ?

this keyword	super keyword
• Refers to current class object.	• Refers to parent class object.
• Used to access current class members.	• Used to access parent class members.
• Can be used to call current class constructor.	• Can be used to call parent class constructor.
• Cannot be used to access parent class members directly.	• Cannot be used to access current class members directly.
• Example: this.name, this(), this.method()	• Example: super.name, super(), super.method()

48. What is the difference between method and constructor ?

Constructor	Method
• Same name as class.	• Can have any name.
• No return type (not even void).	• Must have a return type.
• Called automatically when object is created.	• Called explicitly by the user.
• Used to initialize the object.	• Used to perform a specific task.
• Can be overloaded.	• Can be overloaded and overridden.

49. What is the difference between overloading and overriding ?

Overloading	Overriding
• Occurs in the same class.	• Occurs in parent and child class (in inheritance).
• Same method name, different parameters.	• Same method name and same parameters.
• Compile-time polymorphism (static binding).	• Runtime polymorphism (dynamic binding).
• Return type may or may not be same.	• Return type must be same (or covariant).

50. What is the difference between == and equals() in Java ?

== (Equality Operator)	equals() Method
• Compares memory addresses (reference).	• Compares content (actual values).
• Used for all data types.	• Defined in the Object class and overridden in many classes.
• For objects, it checks if both references point to the same object.	• Used mainly for objects (e.g., String).
	• For String, it checks if the content is the same.
Example :	

`String s1 = new String("Java");`
`String s2 = new String("Java");`
`System.out.println(s1 == s2); // false`
`System.out.println(s1.equals(s2)); // true`
	`String s1 = "Java";`
	`String s2 = "Java";`
	`System.out.println(s1 == s2); // true`
	`System.out.println(s1.equals(s2)); // true`

51. What is the difference between compile-time and runtime ?

Compile-time	Runtime
• Occurs when the program is being compiled.	• Occurs when the program is executed.
• Syntax errors are checked.	• Logical errors and exceptions occur.
• Handled by the compiler.	• Handled by the JVM.
• **Example:** `int a = ;` (syntax error)	• **Example:** `int a = 10/0;` (ArithmeticException)

52. What is a package in Java ?

A package is a namespace that is used to organize related classes and interfaces. It helps in code organization, access control and avoids name conflicts.

Example :

```
package mypack;
public class Demo {
    public void show() {
        System.out.println("This is a package example");
    }
}
```

Key points :

- Package name is written in lowercase.
- Helps in access control.
- Built-in packages : java.lang, java.util, etc.
- We can create our own package.

53. What is an API in Java ?

API stands for Application Programming Interface. It is a collection of pre-built classes, interfaces and methods provided by Java to perform common tasks. It helps developers to write code easily without creating everything from scratch.

Example : Java provides many built-in APIs such as:

- java.lang (String, Math, System)
- java.util (Scanner, ArrayList, Date)
- java.io (File handling)
- java.net (Networking)
- java.sql (Database connectivity)

54. What is the difference between JDK, JRE and JVM ?

JDK (Java Development Kit)	JRE (Java Runtime Environment)	JVM (Java Virtual Machine)
• Complete development package. • Contains JRE + development tools (javac, java, etc). • Used by developers to develop and run Java programs.	• Provides runtime environment. • Contains JVM + core libraries. • Used to run compiled Java programs. • Does not contain development tools.	• Executes Java bytecode. • Converts bytecode to machine code (using JIT). • Provides platform independence. • Part of JRE.

55. What is bytecode in Java ?

Bytecode is an intermediate code generated by the Java compiler from source code (.java file). It is platform-independent and executed by the JVM.

Example :

```
// Source code
class Test {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Compiled by javac ---------->

```
// Bytecode (.class file)
0: getstatic     #2   // Field java/lang/System.out
3: ldc           #3   // String Hello
5: invokevirtual #4   // Method println
8: return
```

56. What is the difference between checked and unchecked exceptions ?

Checked Exceptions	Unchecked Exceptions
• Checked at compile-time. • Must be handled using try-catch or declared with throws. • **Example**: IOException, SQLException.	• Checked at runtime. • Not mandatory to handle. • Usually caused by programming errors. • **Example**: NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException.

57. What is the difference between String, StringBuffer and StringBuilder ?

String	StringBuffer	StringBuilder
• Immutable (cannot be changed). • Every modification creates a new object. • Slower. • Used for fixed and constant data.	• Mutable (can be changed). • Synchronized (thread-safe). • Slower than StringBuilder. • Used in multi-threaded environment.	• Mutable (can be changed). • Not synchronized (not thread-safe). • Faster than StringBuffer. • Used in single-threaded environment.

Ajeet Yadav

58. What is the difference between StringTokenizer and split() ?

StringTokenizer	split()
• It is a class (java.util.StringTokenizer).	• It is a method of String class.
• Part of Java since early versions.	• Introduced in Java 1.4.
• Used to break a string into tokens based on delimiters.	• Used to split a string into an array of substrings based on regex.
• Uses default delimiters like space, tab, newline (or custom).	• Delimiter is specified as a regular expression.
• returns tokens one by one using hasMoreTokens() and nextToken().	• returns all tokens at once in a String array.

59. What is the difference between ArrayList and LinkedList ?

ArrayList	LinkedList
• Implemented using dynamic array.	• Implemented using doubly linked list.
• Faster for random access (get, set).	• Slower for random access.
• Slower for insertion and deletion (in middle).	• Faster for insertion and deletion (in middle).
• Uses less memory.	• Uses more memory (extra node references).
• Better when frequent read operations are needed.	• Better when frequent insertion and deletion operations are needed.

60. What is the difference between HashMap and Hashtable ?

HashMap	Hashtable
• Part of Java Collections Framework (java.util).	• Part of legacy classes (java.util).
• Not synchronized (not thread-safe).	• Synchronized (thread-safe).
• Allows one null key and multiple null values.	• Does not allow null key or null values.
• Introduced in Java 1.2.	• Introduced in Java 1.0.
• Faster than Hashtable.	• Slower than HashMap.

61. What is the difference between throw and throws ?

throw	throws
• Used to explicitly throw an exception from a method or block.	• Used to declare that a method can throw one or more exceptions.
• Followed by an exception object.	• Followed by exception class names (separated by comma).
• Used inside a method.	• Used in method signature.
• Used to create and throw an exception.	• Used for exception handling delegation.

Example (throw):

```
class Test {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            throw new ArithmeticException("Not eligible");
        }
    }
}
```

Example (throws):

```
class Test {
    public static void check() throws IOException {
        FileReader fr = new FileReader("file.txt");
    }
    public static void main(String[] args) {
        try {
            check();
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

62. What is the difference between import statement and package statement ?

import statement	package statement
• Used to import a class or entire package from another package.	• Used to define the current class in a package.
• Helps to use classes without writing full package name.	• Helps to organize classes into packages.
• Written after package statement.	• Must be the first statement in a Java file.
• Example: import java.util.Scanner; import java.util.*;	• Example: package mypack;

63. What is the difference between this and super ?

this keyword	super keyword
• Refers to current class object.	• Refers to parent class object.
• Used to access current class members.	• Used to access parent class members.
• Can be used to call current class constructor.	• Can be used to call parent class constructor.
• Cannot be used to access parent class members directly.	• Cannot be used to access current class members directly.
• Example: this.name, this(), this.method()	• Example: super.name, super(), super.method()

64. What is the difference between instanceof and getClass() ?

instanceof	getClass()
• Used to check whether an object is an instance of a class or interface.	• Returns the actual class of an object at runtime.
• Returns a boolean value (true/false).	• Returns a Class object.
• Also returns true for child class objects.	• Used to get exact class type.
• Example: obj instanceof String	• Example: obj.getClass(); obj.getClass().getName();

65. What is the difference between == and equals() when comparing objects ?

== operator	equals() method
• Compares reference (memory address).	• Compares content (actual values).
• Returns true if both references point to the same object.	• Defined in the Object class and can be overridden.
• Used for both primitive and object comparison.	• Used mainly for object comparison like String, custom classes.
• Example: s1 == s2	• Example: s1.equals(s2)

66. What is the difference between String, StringBuffer and StringBuilder ?

String	StringBuffer	StringBuilder
• Immutable (cannot be changed).	• Mutable (can be changed).	• Mutable (can be changed).
• Every modification creates a new object.	• Synchronized (thread-safe).	• Not synchronized (not thread-safe).
• Slower.	• Slower than StringBuilder.	• Faster than StringBuffer.
• Used for fixed and constant data.	• Used in multi-threaded environment.	• Used in single-threaded environment.
• Example: String s = "Hello";	• Example: StringBuffer sb = new StringBuffer("Hello");	• Example: StringBuilder sb = new StringBuilder("Hello");

67. What is the difference between String, StringBuffer and StringBuilder in terms of synchronization, performance and usage ?

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Synchronization	Not synchronized	Synchronized (thread-safe)	Not synchronized (not thread-safe)
Performance	Slower (new object)	Slower (due to sync)	Faster (no sync)
Usage	For fixed and constant data	For multi-threaded environment	For single-threaded environment

68. What is the difference between abstract class and interface ?

Feature	Abstract Class	Interface
Methods	Can have abstract and concrete methods.	Can have only abstract methods (before Java 8).
Variables	Can have instance variables.	Only public static final variables.
Inhertiance	A class can extend only one abstract class.	A class can implement multiple interfaces.
Constructor	Can have a constructor.	Cannot have a constructor.
Access Modifiers	Can have all access modifiers.	Methods are public by default.
Usage	Used when classes have common properties and behavior.	Used to achieve 100% abstraction and multiple inheritance.

69. What is the difference between extends and implements ?

extends	implements
• Used for class inheritance. • A class can extend only one class (single inheritance). • Used to inherit properties and methods. • Can be used with both abstract and concrete classes. • **Example**: class B extends A { }	• Used for interface implementation. • A class can implement multiple interfaces. • Used to provide the implementation of interface methods. • Can be used only with interfaces. • **Example**: class B implements A, C { }

70. What is method overloading and method overriding ?

Method Overloading

- Same method name, different parameters (number, type or order).
- Occurs in the same class.
- Achieves compile-time polymorphism (static binding).
- Return type may or may not be same.

Example:

```
class Test {
    void show() {
        System.out.println("No parameter");
    }
    void show(int a) {
        System.out.println("With parameter: " + a);
    }
}
```

Method Overriding

- Same method name, same parameters.
- Occurs in parent and child class.
- Achieves runtime polymorphism (dynamic binding).
- Return type must be same (or covariant).
- Used to provide specific implementation.

Example:

```
class Parent {
    void show() {
        System.out.println("Parent class");
    }
}
class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child class");
    }
}
```

71. What is the difference between final, finally and finalize() ?

final	finally	finalize()
• It is a keyword. • Used to make a variable, method or class non-modifiable. • Can be used with variables, methods and classes. • Value cannot be changed once assigned. • Improves security. • **Example:** final int x = 10;	• It is a block. • Used in exception handling (try-catch-finally). • Always executed whether exception occurs or not. • Used to keep cleanup code. • Executes after try and catch blocks. • **Example:** finally { // cleanup code }	• It is a method (from Object class). • Called by Garbage Collector before an object is destroyed. • Used to perform cleanup activities (not recommended). • Its execution is not guaranteed. • Deprecated in recent versions. • **Example:** protected void finalize() { // cleanup code }

72. What is the difference between instanceof and getClass() ?

instanceof	getClass()
• Operator. • Used to check whether an object is an instance of a class or interface. • Returns true or false. • Also returns true for child class objects. • **Example:** obj instanceof String	• Method (from Object class). • Returns the actual Class object of an object at runtime. • Used to get exact class type. • Does not return true for child class objects (only exact class). • **Example:** obj.getClass().getName()

73. What is the difference between super() and this() ?

super()	this()

• Used to call the parent class constructor.
• Must be the first statement in a constructor.
• Used when you want to initialize the parent class part.
• **Example:**
class Child extends Parent {
Child() {
super();
}
}
	• Used to call the current class constructor (constructor chaining).
	• Must be the first statement in a constructor.
	• Used when a class has multiple constructors.
	• **Example:**
	class Test {
	Test() {
	this(10); // calls parameterized constructor
	}
	Test(int x) {
	System.out.println(x);
	}
	}

74. What is the difference between static and non-static (instance) methods ?

Static Method	Non-static (Instance) Method

• Belongs to class.
• Can be called without creating an object (using class name).
• Can directly access only static members.
• Cannot access non-static members directly.
• Used for utility or common functionality.
• **Example:**
class Test {
static void show() {
System.out.println("Static method");
}
}
	• Belongs to object (instance).
	• Must be called using an object.
	• Can access both static and non-static members.
	• Can directly access non-static members.
	• Used for object-specific behavior.
	• **Example:**
	class Test {
	void show() {
	System.out.println("Instance method");
	}
	}

75. What is the difference between interface and abstract class ?

Interface	Abstract Class

• Contains only abstract methods (before Java 8).
• Variables are public, static and final (constants).
• Supports multiple inheritance.
• Used to achieve 100% abstraction.
• All methods are public by default.
• Introduced in Java 1.0.
• **Example:**
`interface Drawable {`
`  void draw();`
`}`
	• Can have both abstract and concrete methods.
	• Can have instance variables.
	• Supports single inheritance.
	• Used when common implementation is needed.
	• Methods can be public, protected or private.
	• Introduced in Java 1.0.
	• **Example:**
	`abstract class Shape {`
	`  abstract void draw();`
	`}`

76. What is the difference between method overloading and method overriding ?

Method Overloading	Method Overriding

• Same method name, different parameters (number, type or order).
• Occurs in the same class.
• Compile-time polymorphism (static binding).
• Return type may or may not be same.
• **Example:**
`class Test {`
`  void show() { }`
`  void show(int x) { }`
`}`
	• Same method name, same parameters.
	• Occurs in parent and child class.
	• Runtime polymorphism (dynamic binding).
	• Return type must be same (or covariant).
	• Child class provides specific implementation.
	• **Example:**
	`class Parent {`
	`  void show() { }`
	`}`
	`class Child extends Parent {`
	`  void show() { }`
	`}`

77. What is the difference between constructor and method ?

Constructor	Method

• Same name as class.
• Called automatically when object is created.
• Used to initialize the object.
• Does not have return type (not even void).
• Can be overloaded.
• Cannot be inherited (but can be called using super()).
• **Example:**
`class Test {`
`  Test() {`
`  System.out.println("Constructor");`
`}`
	• Can have any valid name.
	• Called explicitly by the programmer.
	• Used to perform specific task.
	• Must have a return type (or void).
	• Can be overloaded and overridden.
	• Can be inherited by child class.
	• **Example:**
	`class Test {`
	`  void display() {`
	`  System.out.println("Method");`
	`}`
	`}`

78. What is the difference between length and length() ?

length (for array)	length() (for String)

• It is a public final variable.
• Gives the total number of elements in array.
• Used with arrays.
• Does not use parentheses.
• **Example:**
`int[] arr = new int[5];`
`System.out.println(arr.length); // 5`
	• It is a method.
	• Gives the total number of characters in a string.
	• Used with String class (and some collections).
	• Uses parentheses.
	• **Example:**
	`String s = "Hello";`
	`System.out.println(s.length()); // 5`

79. What is the difference between array and ArrayList ?

Array	ArrayList
• Fixed size (cannot be changed). • Can store both primitive and object types. • Faster (less overhead). • Part of core Java (built-in). • Length is obtained using length property. • **Example:** `int[] arr = new int[5];` `System.out.println(arr.length);`	• Dynamic size (can grow or shrink). • Can store only objects (use wrapper classes for primitives). • Slower than array (due to extra features). • Part of Java Collections Framework (java.util). • Size is obtained using size() method. • **Example:** `ArrayList<Integer> list = new ArrayList<>();` `System.out.println(list.size());`

80. What is the difference between Vector and ArrayList ?

Vector	ArrayList
• Synchronized (thread-safe). • Slower due to synchronization. • Used in multi-threaded environment. • Legacy class (introduced in Java 1.0). • Methods are synchronized by default. • **Example:** `Vector<String> v = new Vector<>();` `v.add("Hello");`	• Not synchronized (not thread-safe). • Faster than Vector. • Used in single-threaded environment (or manual synchronization). • Introduced in Java 1.2. • Methods are not synchronized. • **Example:** `ArrayList<String> list = new ArrayList<>();` `list.add("Hello");`

81. What is the difference between HashSet and LinkedHashSet ?

HashSet	LinkedHashSet
• Does not maintain insertion order. • Faster for basic operations (add, remove, contains). • Uses hashing (HashMap internally). • Allows one null value. • Part of Java Collections Framework (java.util). • **Example:** `HashSet<String> set = new HashSet<>();` `set.add("A"); set.add("B");`	• Maintains insertion order. • Slightly slower than HashSet. • Uses a hash table and linked list (internally). • Allows one null value. • Part of Java Collections Framework (java.util). • **Example:** `LinkedHashSet<String> set = new LinkedHashSet<>();` `set.add("A"); set.add("B");`

82. What is the difference between HashSet and TreeSet ?

HashSet	TreeSet

• Does not maintain any specific order.
• Faster for basic operations.
• Uses hashing (HashMap internally).
• Allows one null value.
• Elements are not sorted.
• **Example:** `HashSet<Integer> set = new HashSet<>();`
`set.add(3); set.add(1); set.add(2);`
	• Maintains elements in sorted (ascending) order.
	• Slower than HashSet.
	• Uses a Tree (Red-Black Tree internally).
	• Does not allow null values.
	• Elements are automatically sorted.
	• **Example:** `TreeSet<Integer> set = new TreeSet<>();`
	`set.add(3); set.add(1); set.add(2);`
	`// Output: 1 2 3`